# Fleet Management · Gestão de Frota

Passo a passo da rota do Motorista (Driver): caminho triste e caminho feliz

> **O projeto e o motivo de sua criação**
>
> O **Fleet Management** é uma API REST de gestão de frota corporativa. Ela controla veículos, motoristas, viagens, abastecimentos, manutenções e incidentes, e responde perguntas do dia a dia: qual veículo está disponível, quem está dirigindo o quê, quanto cada veículo gasta e o que aconteceu em cada viagem, com histórico auditável.
>
> Foi criado como **avaliação técnica de back-end**, para demonstrar num sistema real: regras de integridade impostas no próprio banco (procedures, triggers e CHECKs), autenticação em camadas (API key + JWT) com permissões por papel, integração com uma API externa real (ViaCEP), erros padronizados (RFC 7807) e documentação Swagger. Stack: NestJS, Prisma e PostgreSQL.

## Como ler este documento

Primeiro o **caminho triste**: o que acontece quando o motorista (ou alguém no lugar dele) tenta algo errado, e como o sistema se protege. Depois o **caminho feliz**: o passo a passo completo de uma viagem bem-sucedida. No final, as regras de negócio do sistema e as regras dos outros cargos.

Todas as respostas de erro seguem o padrão RFC 7807 ( `application/problem+json` , com `type` , `title` , `status` e `detail` ). Os valores entre < > são substituídos pelos dados reais; nenhuma chave ou senha aparece aqui.

## Parte 1 · Caminho triste (o que dá errado)

### 1.1 Entrada no sistema

| Ação do motorista | Resposta | Por quê |
|---|---|---|
| Login sem o header `x-api-key` | 401 | O login exige API key **e** e-mail e senha. |
| Login com senha errada | 401 | Credenciais inválidas; a mensagem não revela qual campo falhou. |
| Tentar se cadastrar ( `POST /auth/signup` ) | 404 | Não existe cadastro público. A conta é criada por ADMIN ou gerente. |
| Chamar qualquer rota sem o JWT | 401 | As rotas de negócio exigem `Authorization: Bearer` . |
| Usar a conta depois de ser desativada (soft delete) | 401/403 | Conta inativa não autentica, mesmo com JWT antigo. |

### 1.2 Permissões e escopo

| Ação do motorista | Resposta | Por quê |
|---|---|---|
| `GET /users` ou criar veículo ( `POST /vehicles` ) | 403 | O papel DRIVER não tem essas permissões. |

| | | |
|---|---|---|
| Criar viagem com o `driverId` de outro motorista | **403** | Motorista só age sobre o próprio perfil. |
| Ler, iniciar ou finalizar a viagem de outro motorista | **404** | Para o motorista, o dado alheio simplesmente não existe (evita vazar que existe). |
| Apagar de vez ( `DELETE .../permanent` ) | **403** | Deleção total é exclusiva do ADMIN. |
| Tentar mudar o próprio papel ( `PATCH /users/:id/role` ) | **403** | Só ADMIN troca papel, e ninguém muda o próprio. |
| Usar `PUT` em qualquer recurso | **404** | As rotas `PUT` foram removidas; só existe `PATCH` . |

### 1.3 Regras da viagem

| Ação do motorista | Resposta | Por quê |
|---|---|---|
| Criar viagem com CEP inexistente ou mal formado | **400** | Os CEPs de partida e destino são validados no ViaCEP. |
| ViaCEP fora do ar ou lento | **502/504** | Falha da API externa nunca é confundida com "CEP inválido". |
| Enviar `startKm` ao criar a viagem | **400** | O km inicial é o hodômetro do veículo, definido pelo servidor. |
| Criar viagem com veículo em uso, em manutenção, fora de serviço ou inativo | **409** | Só veículo disponível pode ser usado. |
| Motorista com CNH vencida, inativo ou que já tem viagem ativa | **409** | Regras aplicadas no banco (procedure e triggers). |
| Iniciar com body (ex.: `currentMileage` ) | **400** | O `start` não recebe body; o status já valida. |
| Iniciar viagem que não está PLANNED (ex.: já iniciada) | **409** | Só PLANNED vira IN_PROGRESS. |
| Finalizar com `endKm` 0, negativo ou acima de 100000 | **400** | `endKm` são os km rodados: de 1 a 100000 por viagem. |
| Finalizar viagem que não está IN_PROGRESS | **409** | Só uma viagem em andamento pode ser concluída. |
| Cancelar viagem já concluída | **409** | Concluída ou cancelada não volta atrás. |

### 1.4 Abastecimento e incidente

| Ação do motorista | Resposta | Por quê |
|---|---|---|
| Abastecer enviando `mileage` | **400** | O campo não existe mais; o abastecimento não altera o hodômetro. |
| Abastecer um veículo que está em viagem em andamento de outro motorista | **409** | Com viagem ativa, o abastecimento só vale para o motorista dela. |
| Litros ou preço por litro zero/negativo, combustível inválido | **400** | Validação de DTO e da procedure. |

| | | |
|---|---|---|
| Registrar incidente com tipo ou gravidade inválidos, foto fora do padrão | 400 | Tipos: ACCIDENT, MECHANICAL_FAILURE, OTHER; gravidades: LOW, MEDIUM, HIGH; foto JPEG/PNG/PDF até 10 MB. |
| Muitas requisições em pouco tempo | 429 | Limite de requisições (rate limiting). |

## Parte 2 · Caminho feliz (passo a passo)

**Cenário:** a motorista Ana recebe a conta, faz login, planeja uma viagem, inicia, abastece, registra a chegada e confere o resultado.

### Passo 0 · Conta criada por ADMIN ou gerente

`POST /users` com papel DRIVER e o bloco `driver` (CNH e validade). O servidor gera a API key e a devolve **uma única vez**; o gestor a entrega à motorista. O perfil de motorista nasce junto com o usuário.

### Passo 1 · Login

`POST /auth/login` com o header `x-api-key` e o body abaixo. 200

```
{ "email": "<email>", "password": "<senha>" }
```

A resposta traz o `accessToken` (JWT) e o `user`, com `permissions` e `driverId`. Daqui em diante: `Authorization: Bearer <accessToken>`.

### Passo 2 · Conferir o próprio perfil

`GET /users/me` devolve papel, permissões efetivas e o `driverId` que será usado nos próximos passos. 200

### Passo 3 · Planejar a viagem

`POST /trips` 201

```
{ "driverId": "<driverId>", "vehicleId": "<id do veículo disponível>",
  "startLocation": "01310-100", "endLocation": "20040-020" }
```

Os CEPs são validados no ViaCEP. A viagem nasce **PLANNED**, o veículo passa a **IN_USE** e o km inicial é o hodômetro atual do veículo (sem o motorista informar km).

### Passo 4 · Iniciar a viagem

`PATCH /trips/:id/start`, **sem body**. 200 A viagem vai para **IN_PROGRESS**.

### Passo 5 · Abastecer no caminho

`POST /refuelings` 201

```
{ "vehicleId": "<id>", "driverId": "<driverId>",
  "litersAdded": 10.5, "costPerLiter": 5.12, "fuelType": "GASOLINE" }
```

O custo total é calculado pelo sistema. O abastecimento guarda o hodômetro do momento e **não altera** o hodômetro do veículo.

### Passo 6 · Registrar um incidente (se houver)

`POST /incidents` (multipart/form-data) com veículo, motorista, tipo, gravidade, descrição e, opcionalmente, uma foto. 201

### Passo 7 · Finalizar a viagem

`PATCH /trips/:id/end` 200

```
{ "endKm": 120, "endLocation": "20040-020" }
```

`endKm` são os **km rodados**. O servidor soma ao hodômetro do veículo (ex.: 82000 + 120 = 82120), conclui a viagem (**COMPLETED**) e devolve o veículo para **AVAILABLE**. A resposta traz o hodômetro final e o `distanceKm`.

### Passo 8 · Consultar o resultado

`GET /trips/:id` mostra a viagem concluída; `GET /trips` lista **somente as viagens da própria motorista**. `200`

### Alternativa · Cancelar

Se a viagem não vai acontecer: `PATCH /trips/:id/cancel` (permitido antes de concluir). A viagem fica **CANCELLED** e o veículo é liberado.

### Resumo do ciclo

| Estado | Como chega | Veículo | Hodômetro |
|---|---|---|---|
| PLANNED | `POST /trips` | IN_USE | Igual ao do veículo (km inicial provisório) |
| IN_PROGRESS | `PATCH .../start` | IN_USE | Não muda |
| COMPLETED | `PATCH .../end` | AVAILABLE | Soma os km rodados |
| CANCELLED | `PATCH .../cancel` | AVAILABLE | Não muda |

# Parte 3 · Principais regras de negócio

## Regras gerais do sistema

- **Sem cadastro público.** Toda conta nasce em `POST /users`, feito por ADMIN ou por quem tem a permissão `USER_CREATE`. O motorista nasce junto com o usuário.
- **Acesso em duas camadas.** Login com API key + e-mail + senha; depois JWT. A API key é guardada apenas como hash e mostrada uma única vez. Renovar a própria chave exige a senha.
- **Permissões por papel e por delegação.** Cada papel tem um conjunto de permissões; o ADMIN pode conceder ou retirar permissões individuais de um usuário.
- **Regras no banco, não só na aplicação.** O ciclo da viagem, abastecimento e incidente rodam em procedures; triggers e CHECKs bloqueiam dados inválidos mesmo em escrita direta.
- **Hodômetro só muda por viagens.** Cadastro do veículo define o valor inicial; depois, só o fim de uma viagem soma km. Editar o hodômetro de um veículo em uso é bloqueado.
- **Uma viagem ativa por vez.** Um veículo e um motorista só podem ter uma viagem ativa cada; veículo em manutenção, fora de serviço, em uso ou inativo não viaja; CNH vencida bloqueia o motorista.
- **Endereços validados.** Os CEPs de viagem e o local inicial do veículo passam pelo ViaCEP; falha da API externa é 502/504, nunca "CEP inválido".
- **Soft delete e restauração.** Apagar comum é reversível (o registro fica inativo); restaurar é permissão própria. Deleção total é exclusiva do ADMIN e bloqueada quando há dependências (por exemplo, veículo com viagens); abastecimentos de um veículo apagado saem em cascata.
- **Auditoria imutável.** Mudanças em viagens, abastecimentos e incidentes geram registro em `audit_logs`, que não aceita alteração nem exclusão (só o autor pode ser anonimizado quando o usuário é apagado de vez).
- **Erros padronizados** (RFC 7807), limite de requisições e logs sem dados sensíveis.

## Regras por cargo

| Cargo | O que pode | O que não pode |
|---|---|---|
| **Motorista** (DRIVER) | Fazer login; ver o próprio perfil; criar, iniciar, finalizar e cancelar as **próprias** viagens; registrar abastecimentos e incidentes com o **próprio** perfil; consultar apenas os próprios dados. | Ver dados de outros motoristas (recebe 404); cadastrar veículos ou usuários; trocar papel; apagar de vez. |
| **Gerente de Frota** (FLEET_MANAGER) | Gerir veículos, manutenções, viagens, abastecimentos e incidentes de toda a frota; ver indicadores (analytics); criar contas de **motorista**; usar as permissões que o ADMIN delegar (ex.: listar usuários). | Criar gerente ou ADMIN; trocar papéis; mexer em contas ADMIN; deleção total; reemitir API key de outro usuário. |
| **Administrador** (ADMIN) | Tudo: criar contas de qualquer papel; promover usuários; delegar e retirar permissões; reemitir API key; deleção total; consultar auditoria e todos os dados. | Rebaixar papel (a troca só sobe, e ninguém muda o próprio); desativar ou apagar outro ADMIN ou a si mesmo. |

## Regras de proteção de contas

- **Contas ADMIN são protegidas:** quem não é ADMIN nunca altera uma conta ADMIN, mesmo com permissão delegada.

- **A troca de papel só sobe** (por exemplo, DRIVER para FLEET_MANAGER). Uma promoção errada só se desfaz desativando a conta.
- **Motorista sai de circulação** pelo soft delete; a conta desativada perde o acesso na hora.